《物种起源》复刻项目 - 性能测试报告

测试人员：ADLD
测试日期：2026.04.10
测试版本：v0.1（核心数值系统完成版）

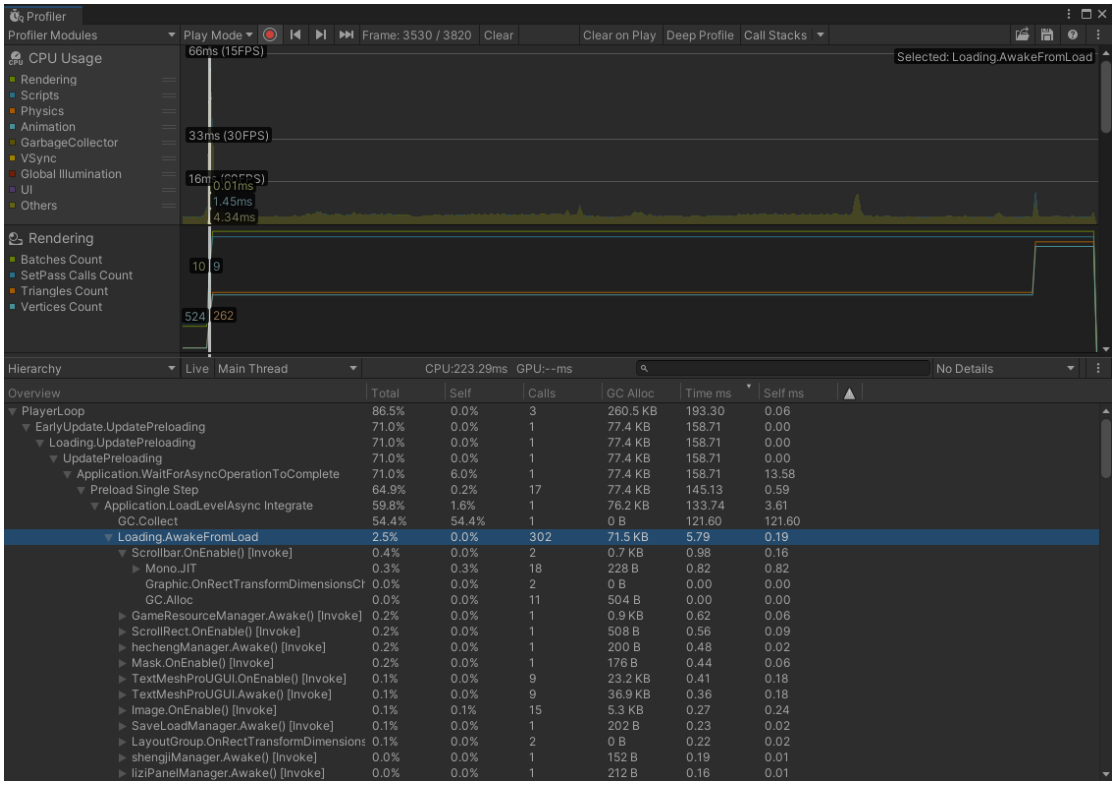
测试环境

项目	配置
测试设备（编辑器）	Win10，i7-12700F，RX 7800XT，32G 内存
测试设备（真机）	荣耀 magic 6，Android 15
Unity 版本	Unity 2022.3.62f1c1
测试工具	Unity Profiler，Memory Profiler
测试场景	游戏主场景（挂机 + 交互）

测试场景与操作

场景 1：游戏启动时刻
场景 2：挂机 10 分钟（无操作，验证资源生产循环稳定性）

测试结果&关键性能指标
场景 1： UnityProfiler CPU Usage

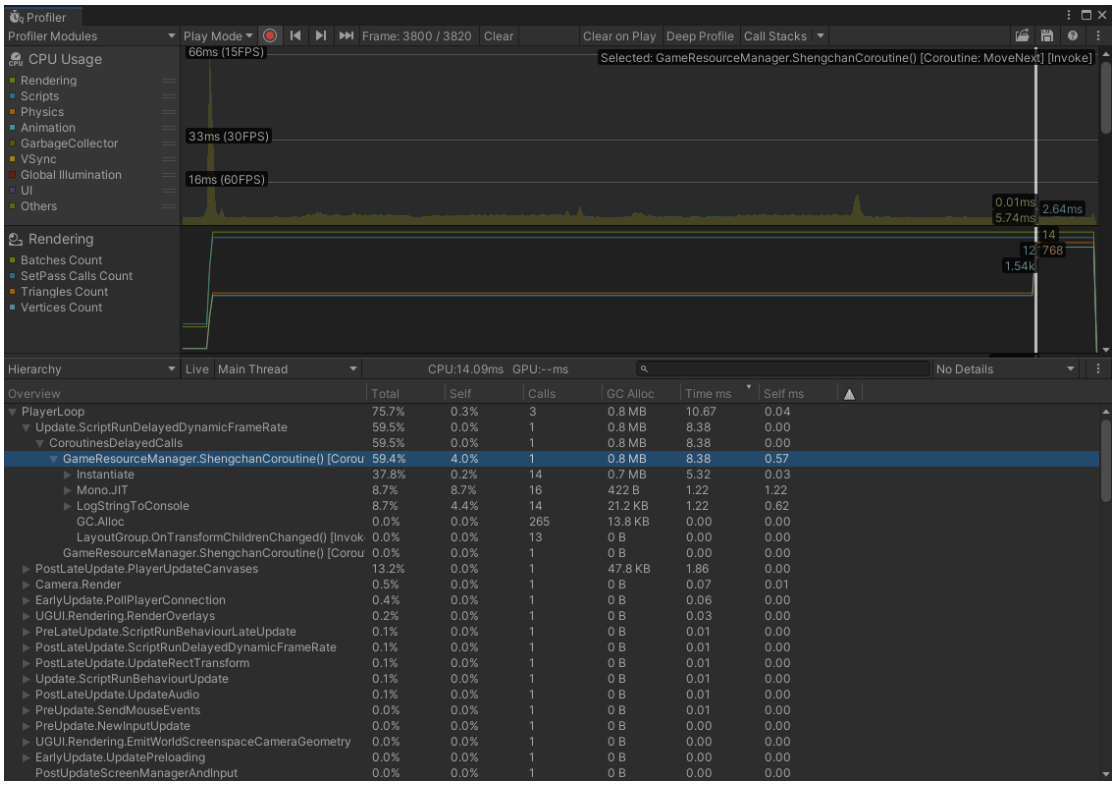


启动跳转场景时，帧率探底至 15fps，其他时刻均高于 60fps。
GC Alloc：Awake()初始化过程占用 71KB，占 Player Loop (260KB)的接近 1/4，其中主要是

文本框从禁用变为启动，以及被加载时的消耗

Time ms：GC.Collect 占用 121ms，占 Player Loop(193ms)的 3/5，

结论：属于场景切换的正常开销，仅出现一次，不影响运行时流畅度。

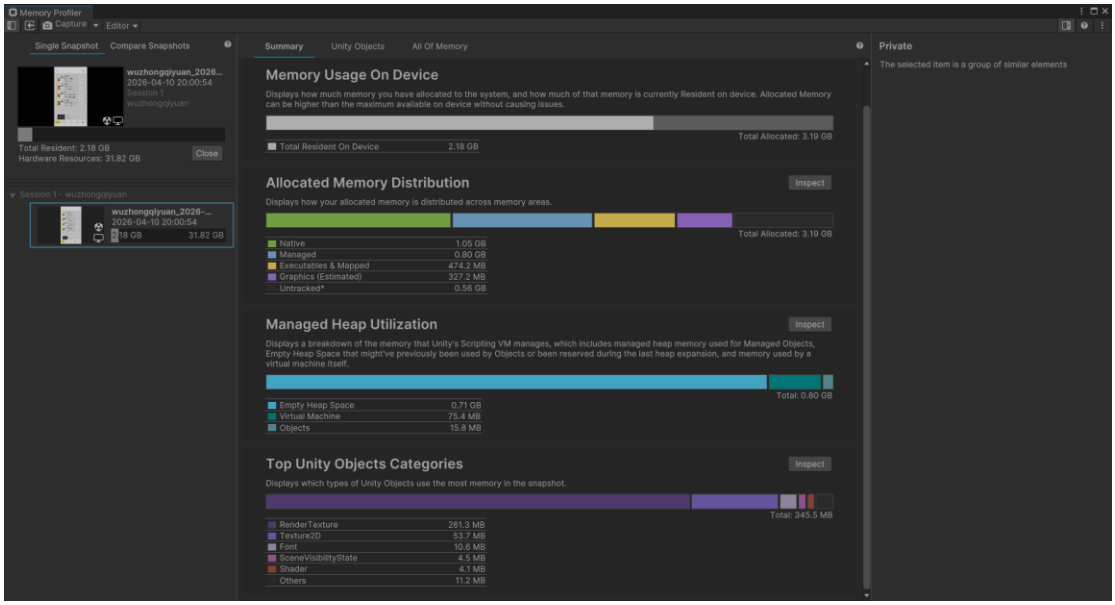


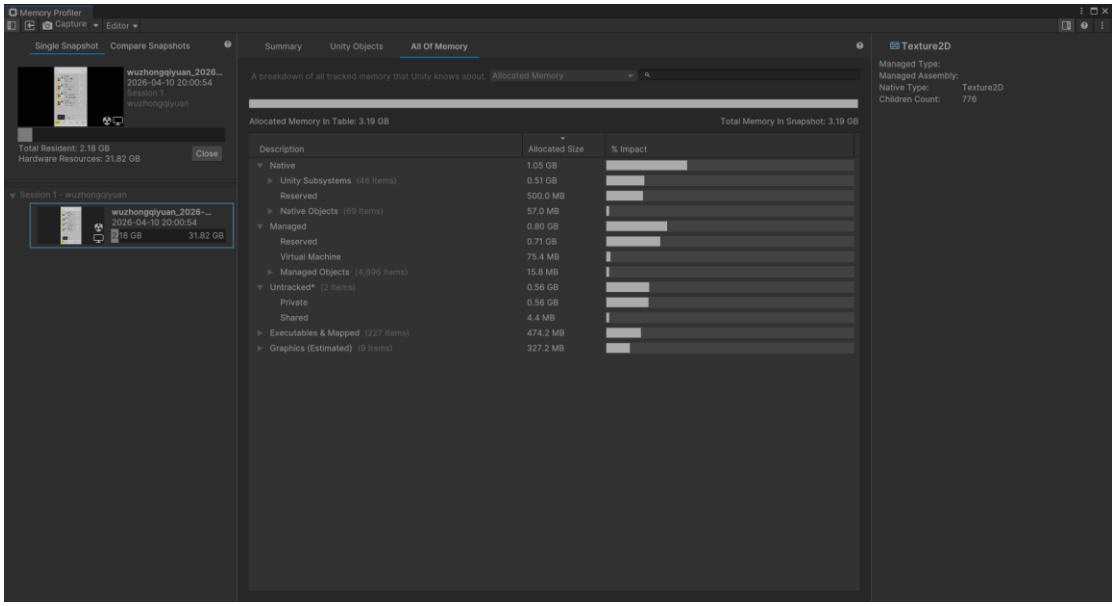
启动后第 3800 帧，有一个明显波峰。

GC Alloc 和 Time ms 的性能开销主要来源均为资源管理器脚本的生产协程方法.

结论：属于脚本加载时的正常开销，仅出现一次，不影响运行时流畅度。

MemoryProfiler （Editor 环境）





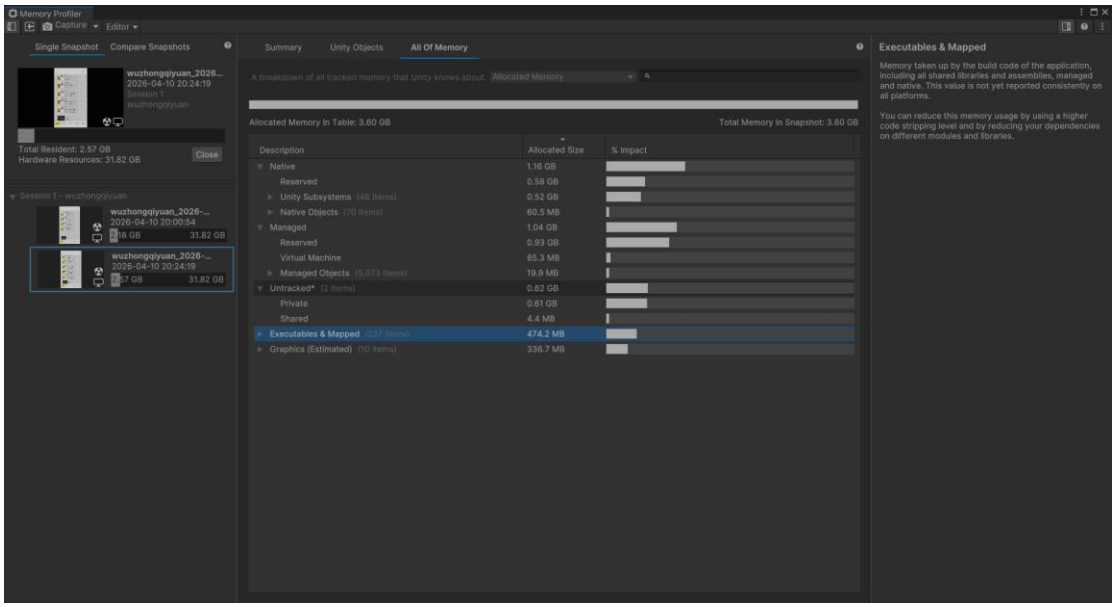
Unity Subsystems 和 Reserved 合计超过 1GB，需要真机包在进行一次测试

托管堆总分配 0.80 GB，其中存活对象仅 15.8 MB，剩余为空闲空间。

结论：Editor 环境内存数据受工具自身影响较大，需以真机测试为准。重点关注 RenderTexture 是否泄漏。

总结：在 Unity Editor 中测试，总内存占用 3.19 GB，Editor 自身开销占比较大（Native Subsystems、Untracked 等）。重点发现 **RenderTexture 占用 261 MB**，需在真机上验证并优化。托管堆存活对象仅 15.8 MB，无内存泄漏。

场景 2 MemoryProfiler（Editor 环境）



游戏运行 24 分钟后内存消耗：

在编辑器环境下持续运行 24 分钟，总内存占用达到 **2.57 GB**，接近中端移动设备（4GB RAM）的内存警戒线。主要问题集中在：

1.Native 内存偏高（1.16 GB），其中 Unity Subsystems 占用 0.52 GB，推测与未卸载的资源或调试开销有关；

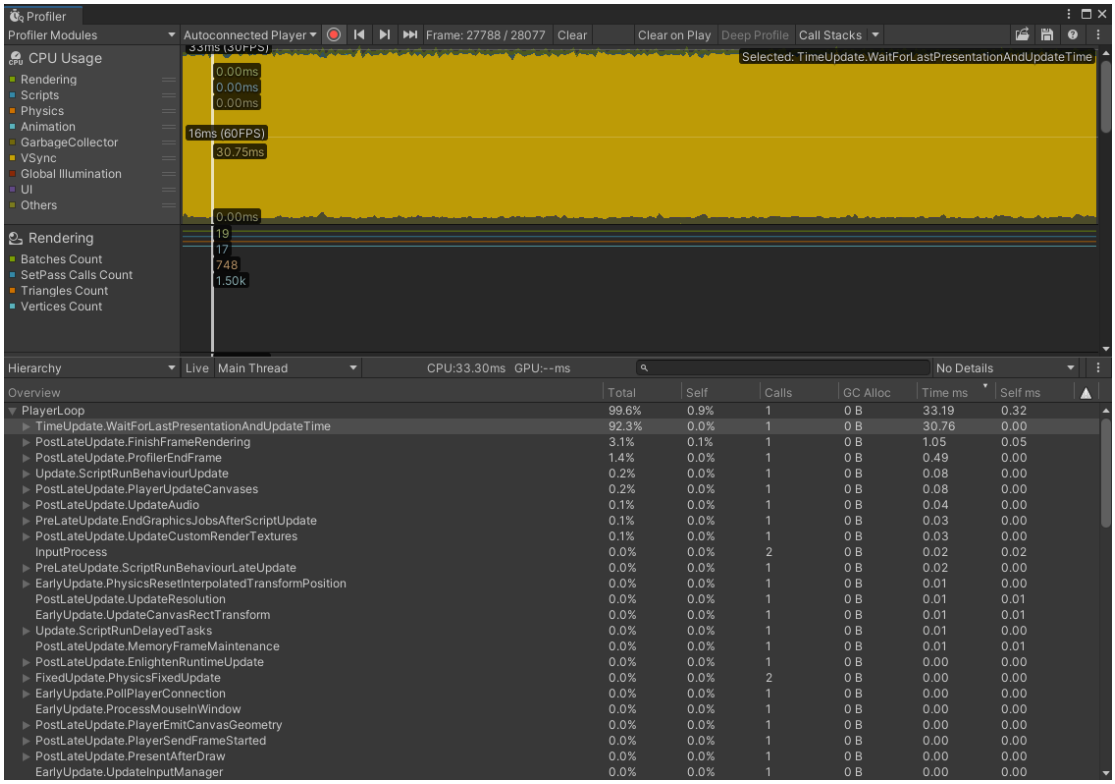
2.Managed 托管堆空闲空间过大（0.93 GB），实际存活对象仅 19.9 MB，说明曾经发生过大量临时对象分配，导致堆膨胀；

3.Untracked 内存 0.62 GB，可能来自第三方插件或底层驱动，需进一步排查；

4.Graphics 预估 337 MB，其中 RenderTexture 仍可能是主要消耗源（需结合细分数据确认）。

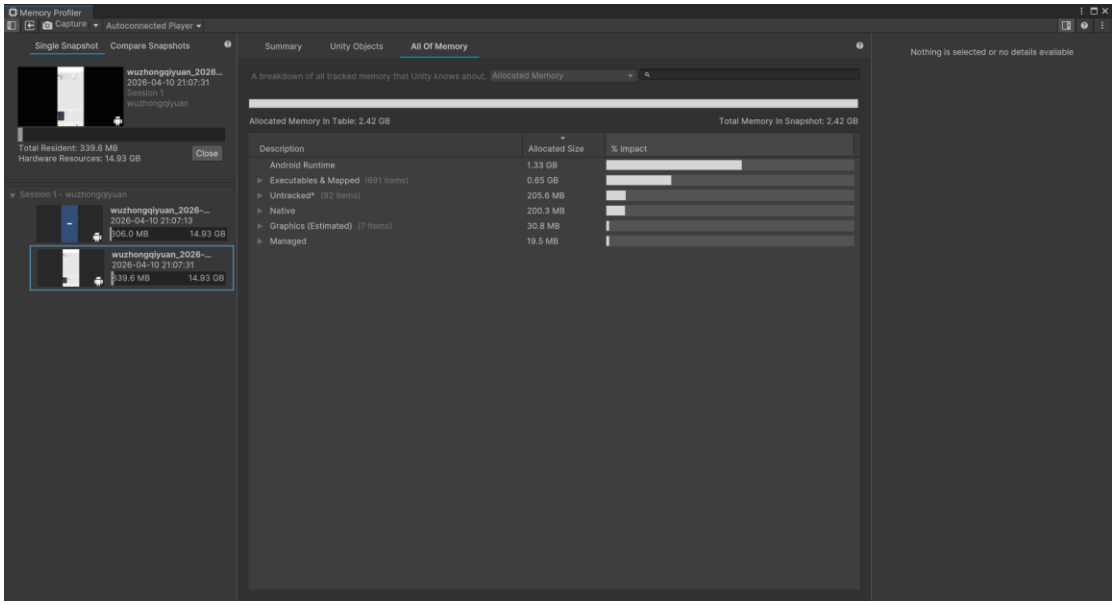
目前无明显内存泄漏（存活对象稳定），但整体内存压力较大。

场景 1 UnityProfiler　真机环境



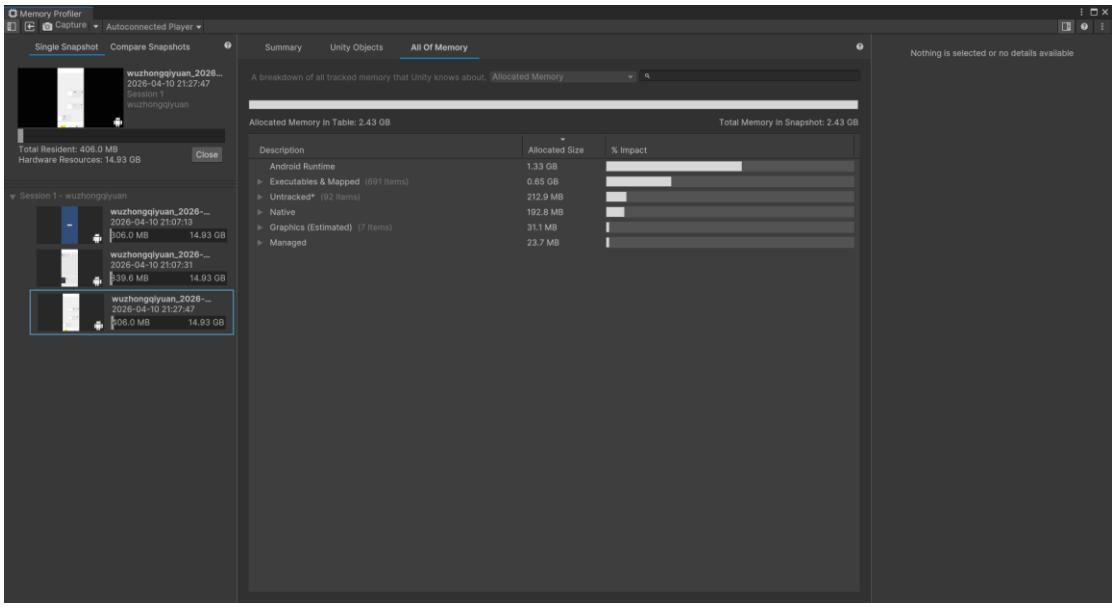
稳定在 30fps 左右，性能主要开销在每秒的资源生产，可以接受。

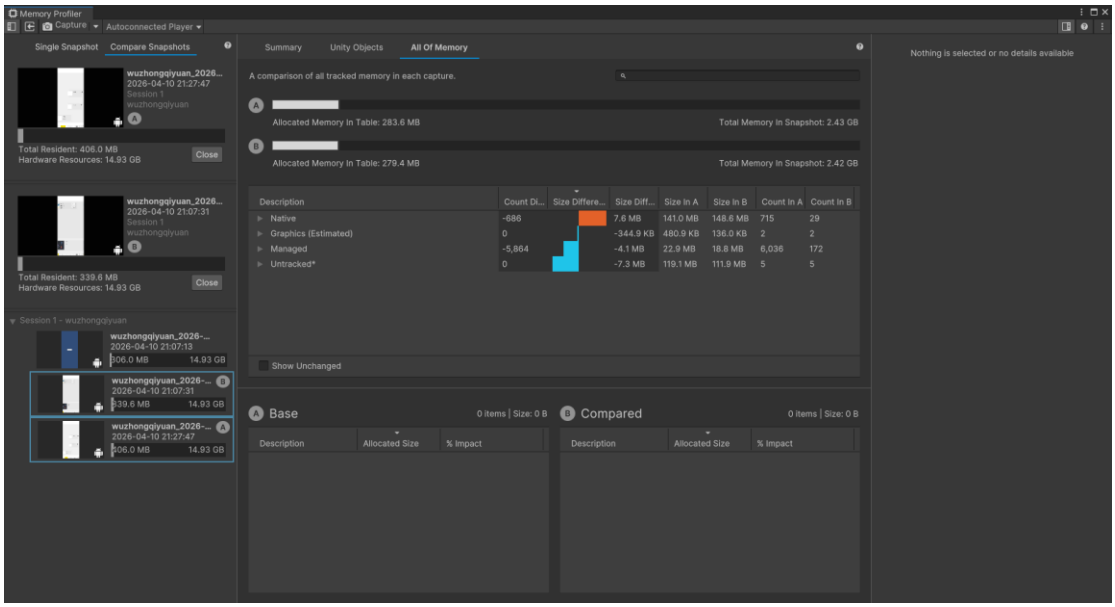
MemoryProfiler 真机环境

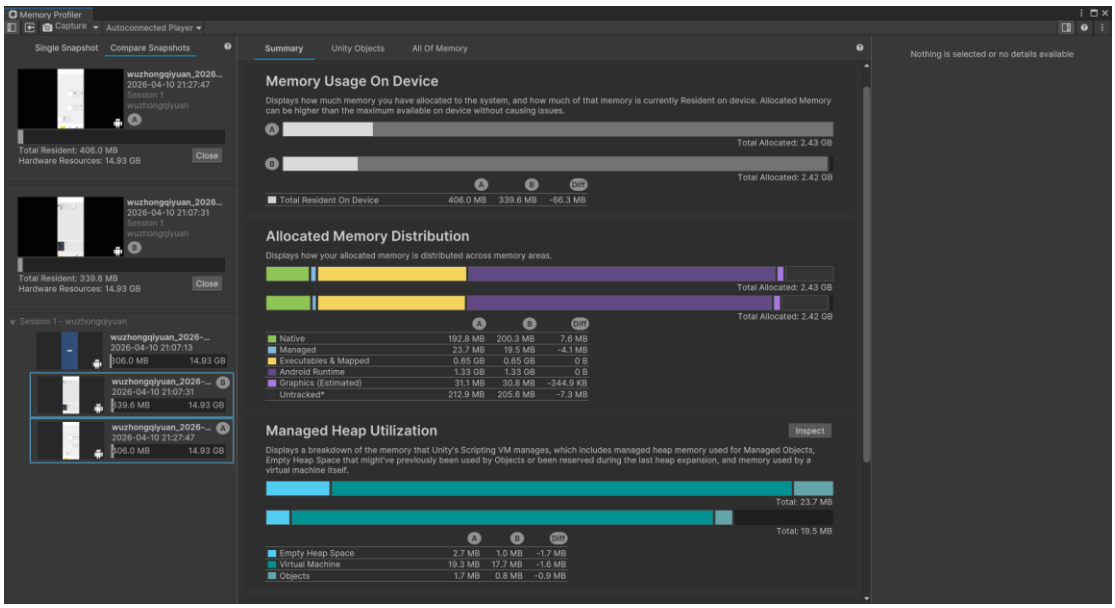


在真机（Android 64 位）上对《物种起源》复刻项目进行内存测试，启动并完成场景跳转后总内存占用 2.42 GB。发现 **Android Runtime 占用高达 1.33 GB**，**Executables 占用 0.65 GB**，为主要内存消耗源。游戏自身资源（Native、Graphics、Managed）合计仅约 250 MB，无内存泄漏。

场景 2 MemoryProfiler 真机环境







对《物种起源》复刻项目进行真机内存测试（运行 20 分钟），总内存占用稳定在 2.43 GB，无内存泄漏。游戏自身资源（Native、Graphics、Managed）仅占约 250 MB，表现健康。主要内存消耗来自 **Android Runtime（1.33 GB）**和 **Executables（0.65 GB）**。

整体总结：

性能达标项

1.游戏运行时帧率稳定在 30 FPS（真机）/ 60+ FPS（Editor）。

2.无内存泄漏，长时间运行内存占用稳定。

3.游戏自身资源（纹理、脚本、网格）管理良好，占用仅约 250 MB。

待优化项

1. Android Runtime 占用 1.33 GB
   优化方式：启用高等级代码裁剪（Managed Stripping Level High），移除未使用的托管 DLL
2. Executables & Mapped 占用 0.65 GB
   优化方式：使用 IL2CPP 的“Faster (smaller) builds”选项，启用 Split Application Binary